

Adaptive Cognitive Budgeting for GPU-Free Language Models on Memory-Constrained Personal Computers

R. Sai Dheeraj
Independent Researcher
Kurnool, Andhra Pradesh, India
16saidheeraj@gmail.com

Abstract—Running large language models (LLMs) locally on ordinary, GPU-free personal computers is attractive for privacy, cost, and offline use, but such machines are constrained simultaneously along several axes — system RAM, CPU core count, memory bandwidth, and thermal headroom — that current inference stacks generally treat as fixed rather than as resources to be actively managed. Existing adaptive-computation techniques either target GPU-resident KV-cache budgets in server settings or regulate reasoning-token length for accuracy/latency trade-offs, but neither line of work adapts the whole inference configuration — quantization level, context window, generation length, KV-cache retention, and CPU thread allocation — jointly to a single machine’s live memory state. I propose Adaptive Cognitive Budgeting (ACB), a control layer above a CPU-based GGUF inference engine that treats these five knobs as one unified cognitive budget per query, combining a lightweight query-complexity estimator with OS telemetry to compute a budget before each request and re-negotiate it during generation when memory pressure crosses a threshold. I report a completed controlled experiment on an Apple M2 system (8 GB unified memory, Llama-3.2-1B-Instruct, CPU-only). Across 20 matched queries per configuration, all three configurations completed without recorded process termination, truncation, or OOM events; ACB’s observed maximum latency was 53.14 s, versus individual observations of 402.42 s (Static-Safe) and 4057.91 s (Static-Capable). Because these figures come from a single trial per configuration on one machine, one model, and one 20-query workload, I present them as bounded, audited observations rather than universal performance claims, and specify in an appendix the repeated-trial and additional-baseline protocol needed to test generality.

Index Terms—*adaptive computation, resource-constrained inference, on-device language models, memory management, quantization, GPU-free inference, edge AI, cognitive budgeting*

I. INTRODUCTION

Large language models (LLMs) are increasingly run outside data centers — on laptops, low-cost desktops, and single-board computers — driven by privacy concerns, offline requirements, and the cost of API access. Projects such as llama.cpp [1] and the GGUF quantized-model ecosystem have made it possible to execute multi-billion-parameter models on CPU-only hardware with no discrete GPU. However, “possible” does not mean “well-managed”: a personal computer with limited RAM must share that memory among the operating system, browser, background services, and the model process itself. The dominant deployment pattern is static: a user or installer selects one quantization level and one context length for a model and uses them for every query, regardless of query complexity or current host memory availability.

This static configuration can be inefficient in both directions. When a query is simple and the machine is relatively idle, the model may retain a context and precision setting intended for

more demanding inputs. When the query is complex or the machine is already under memory pressure, the same configuration can increase memory demand, trigger swapping, or fail with an out-of-memory condition.

A CPU-bound LLM deployment has several independently adjustable knobs that jointly determine its memory and compute footprint: weight quantization precision, maximum admitted context, maximum generated tokens, KV-cache retention, and CPU thread count. Individually, these dimensions have been studied — quantization for accuracy/size trade-offs [2], [3], KV-cache eviction for long-context memory pressure [4], [5], and reasoning-token budgets for latency/accuracy trade-offs [8]–[10]. What is missing is a layer that treats these knobs as a single budget recomputed per query from both query complexity and host-resource state.

I call this joint allocation problem Adaptive Cognitive Budgeting (ACB). This paper makes the following contributions:

- I formalize the joint allocation of quantization, context window, generation length, KV-cache retention, and thread count for CPU-only LLM inference as a per-query budget-allocation problem.
- I present an ACB architecture consisting of a complexity estimator, resource telemetry monitor, budget controller, and re-negotiation hook above an otherwise unmodified GGUF/llama.cpp inference engine.
- I present an analytical memory model for GGUF quantization and KV-cache growth and a reproducible CPU-only evaluation protocol.
- I report a completed controlled experiment on an 8 GB Apple Silicon system, separating measured observations from analytical quantities, and specify in Appendix B the additional trials needed to test generality.

II. RELATED WORK

A. GPU-Resident Memory Budgeting for Inference

A substantial body of recent work adapts memory usage for LLM inference, but much of it assumes a GPU is present and the scarce resource is VRAM. ARKV [4] performs data-driven KV-cache eviction to keep long-context inference within a fixed GPU memory budget. APEX [5] schedules work across CPU and GPU under GPU memory constraints. PowerInfer [6] exploits activation sparsity to reduce GPU memory demand on consumer hardware. NeuroFlux [7] adapts CNN training under constrained accelerator memory. ACB differs in that it assumes

no discrete GPU and treats shared system memory as the principal resource.

B. Reasoning-Token and Computation Budgeting

AdaCtrl [8] and SelfBudgeter [9] adapt reasoning-token budgets based on estimated problem difficulty. Hierarchical adaptive budgeting [10] allocates computation at multiple levels, while cognitive-load-aware inference [11] and input-adaptive computation allocation [12] also address per-query computation budgets. These approaches primarily budget generation or reasoning computation; ACB additionally considers quantization, context admission, KV-cache retention, and CPU threads under host-memory constraints.

C. Edge and Resource-Constrained Deployment

The Cognitive Edge Computing survey [13] synthesizes quantization, sparsity, distillation, and context-compression techniques for constrained devices. Adaptive checkpointing for parameter-efficient fine-tuning [14] similarly demonstrates the value of memory-aware policies. ACB extends the resource-budgeting perspective to CPU-only inference on a general-purpose personal computer sharing memory with unrelated applications.

D. Positioning

To the best of my knowledge, ACB is designed as a single control layer that jointly considers quantization, context window, generation length, KV-cache retention, and CPU thread allocation using both query-complexity estimation and live host telemetry for GPU-free personal-computer inference. ACB is a control layer rather than a new model architecture or training procedure.

III. PROBLEM FORMULATION

Consider a personal computer with total system memory M , of which $F(t)$ is free at time t . For an incoming query q , the inference engine selects

$$c = (p, w, g, k, n) \quad (1)$$

where p is weight quantization precision, w is the admitted context window, g is the maximum generation length, k is the KV-cache retention policy, and n is the number of CPU threads. Each configuration induces an approximate memory footprint

$$\hat{m}(c) \approx m_w(p) + m_{kv}(p, w, k) + m_{act}(p, n) \quad (2)$$

where m_w , m_{kv} , and m_{act} are the weight, KV-cache, and activation terms, and $\ell(c, q)$ is a query-dependent latency. The controller seeks a configuration that maximizes a quality proxy subject to a memory-safety constraint,

$$\max_c \text{Quality}(c, q) \text{ s.t. } \hat{m}(c) \leq \alpha F(t) \quad (3)$$

where $\alpha \in (0, 1)$ leaves headroom for the operating system and other applications; α is realized concretely in Section IV-C as the memory-derived cap on context and generation length, and in the KV-policy switch to a sliding window when the estimated cache would exceed the configured allowance. Because the

exact quality function is unavailable before generation and host state changes continuously, ACB implements a fast rule-based approximation rather than solving the optimization problem directly.

IV. ADAPTIVE COGNITIVE BUDGETING ARCHITECTURE

ACB is organized as four cooperating components between the application and an otherwise unmodified inference engine: a complexity estimator, resource telemetry monitor, budget controller, and re-negotiation hook. Fig. 1 shows how they fit together.

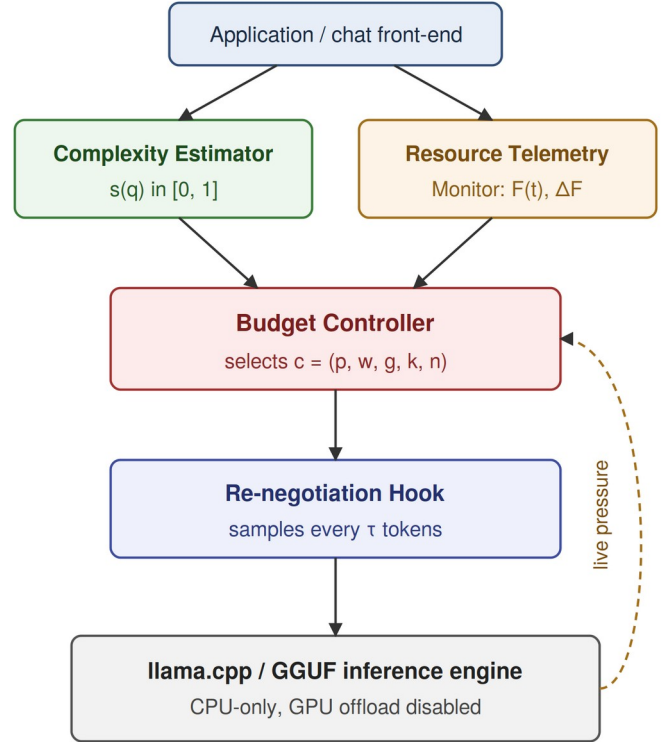


Fig. 1. ACB architecture. Query complexity and live host telemetry feed the budget controller, which configures the engine through the re-negotiation hook; live pressure is fed back during generation.

A. Complexity Estimator

Before dispatch, a lightweight estimator produces a scalar complexity score $s(q) \in [0, 1]$ from input length, presence of code or structured data, simple sub-question indicators, and short conversation history. The estimator is deliberately model-free and uses inexpensive lexical and length-based features. The present work does not separately validate $s(q)$ against ground-truth query difficulty or measured memory demand; Appendix B proposes an ablation to isolate its contribution.

B. Resource Telemetry Monitor

A background monitor samples host memory state at a fixed interval. On macOS, the implementation uses system memory statistics and the `memory_pressure` utility. The monitor exposes

current memory availability and a pressure trend to the controller.

C. Budget Controller

The controller maps $s(q)$ and host state to configuration c . Precision is selected from a small ladder. Context and generation budgets scale with query complexity subject to a memory-derived cap. KV-cache retention can switch to a sliding policy when the estimated cache would exceed the configured memory allowance. Thread count is constrained when memory pressure worsens. Fig. 2 summarizes this mapping.

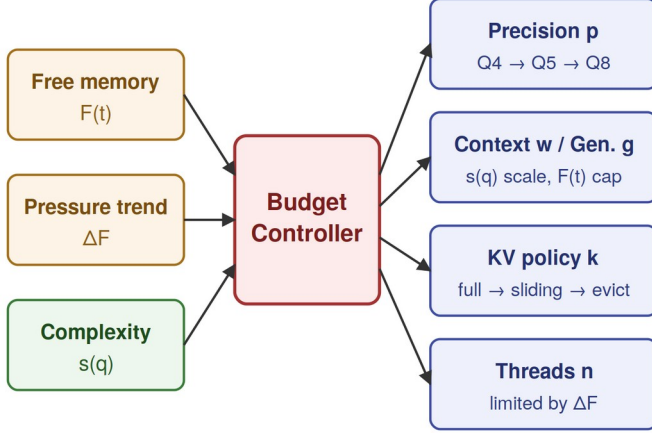


Fig. 2. Budget-controller mapping. Host memory state gates the available resource budget, while query complexity scales context and generation within that budget.

D. Re-Negotiation Hook

Because generation can take seconds or minutes, ACB periodically re-samples host state during generation. If a hard pressure threshold is crossed, the implementation can stop generation and mark the output as truncated rather than allowing uncontrolled failure.

E. Per-Query Control Procedure

The implemented control sequence follows the rule-based procedure in Algorithm 1; Fig. 3 gives the equivalent flowchart.

Algorithm 1 Adaptive Cognitive Budgeting per Query

```

Require: Query q, telemetry monitor T
1:  $s \leftarrow \text{EstimateComplexity}(q)$ 
2:  $(F, \Delta F) \leftarrow T.\text{sample}()$ 
3:  $p \leftarrow \text{SelectPrecision}(F)$ 
4:  $(w, g) \leftarrow \text{ScaleContextAndGeneration}(s, F, p)$ 
5:  $k \leftarrow \text{SelectKVPolicy}(w, F, p)$ 
6:  $n \leftarrow \text{SelectThreads}(\Delta F)$ 
7:  $c \leftarrow (p, w, g, k, n)$ 
8: Dispatch c to the inference engine
9: for each generated batch of  $\tau$  tokens do
10:    $(F, \Delta F) \leftarrow T.\text{sample}()$ 
11:   if pressure > hard threshold then
12:     stop; mark output truncated
13:     break
14: return output, c, truncation flag
  
```

Fig. 4 takes the same procedure and follows the data itself rather than the components, from the raw query text through to one audited row of the completed dataset.

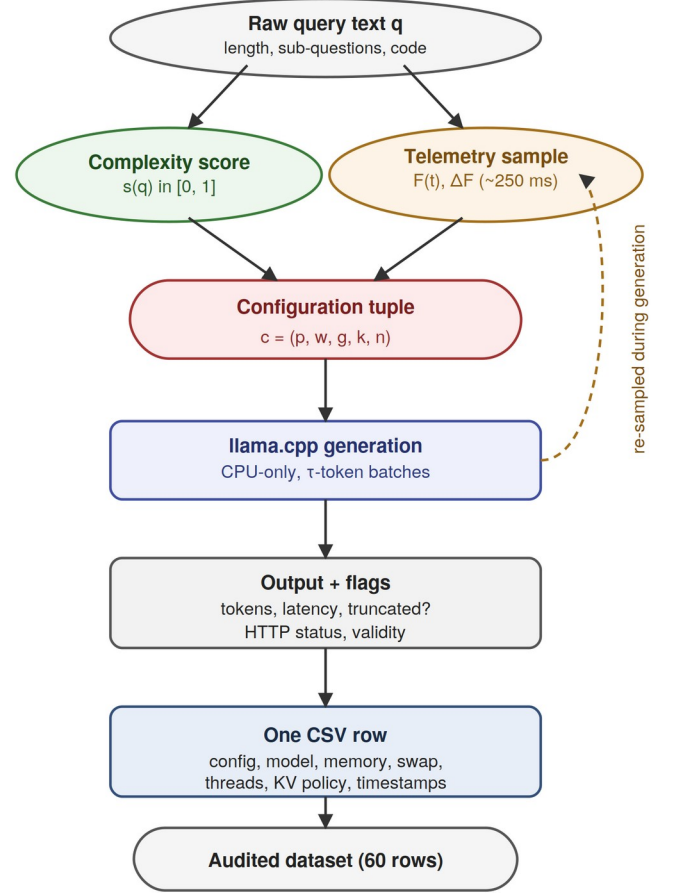


Fig. 4. Per-query data flow. The query text yields a complexity score and combines with a telemetry sample into a configuration tuple; llama.cpp consumes that tuple and the resulting output becomes one row of the audited CSV.

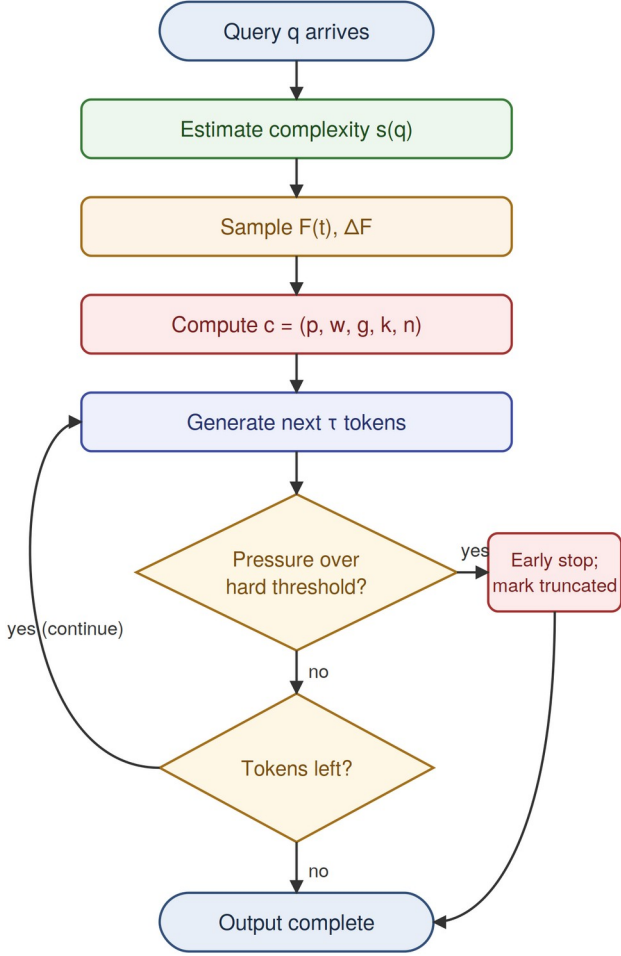


Fig. 3. Per-query ACB control flow. The controller establishes an initial budget and the re-negotiation hook samples host state during generation.

F. Implementation

The reference implementation wraps llama.cpp server mode through its HTTP API. The complexity estimator and controller operate as a thin Python proxy. Context and generation limits are passed through llama.cpp runtime parameters, and CPU-only execution is enforced by disabling GPU offload.

V. TESTBED, ANALYTICAL MODEL, AND EXPERIMENTAL PROTOCOL

A. Testbed

The experiment was conducted on an Apple M2 MacBook Pro with 8 GB of unified memory and 256 GB of storage running macOS. GPU/Metal offload was explicitly disabled using `--n-gpu-layers 0`, so inference ran in the CPU-only regime.

B. Candidate Models and Analytical Memory Model

The broader study considers Llama-3.2-1B-Instruct, Qwen2.5-3B-Instruct, and Llama-3.1-8B-Instruct. The estimated GGUF weight footprints in Table I are design-time analytical estimates and are not used as empirical measurements in Section VII. Fig. 5 plots these estimates against the 8 GB of

system memory on the test machine; only the 8B model at Q8_0 exceeds it.

TABLE I. CANDIDATE MODELS AND ANALYTICALLY ESTIMATED WEIGHT MEMORY

Model	Params	Q4_K_M (est.)	Q8_0 (est.)
Llama-3.2-1B-Instruct	1B	~0.8 GB	~1.3 GB
Qwen2.5-3B-Instruct	3B	~2.0 GB	~3.3 GB
Llama-3.1-8B-Instruct	8B	~4.9 GB	~8.5 GB

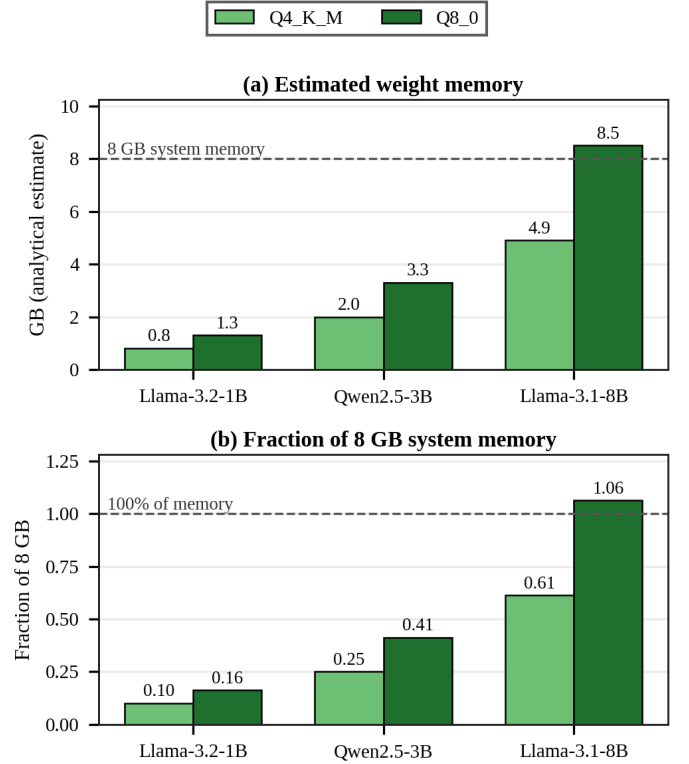


Fig. 5. Analytically estimated GGUF weight memory for the three candidate models at Q4_K_M and Q8_0 (from Table I): (a) in GB and (b) as a fraction of the 8 GB test machine. These are design-time estimates, not measurements.

KV-cache memory is modeled as

$$m_{kv} \approx 2 \times L \times H \times d_h \times w \times b \quad (4)$$

where L is the number of transformer layers, H the number of key/value heads, d_h the head dimension, w the context length, and b the bytes per cached element. This model motivates the context-budget control but is not presented as a measured memory quantity. For the 3B-class model considered analytically, it gives an approximate 150–250 MB increase per additional 1,000 context tokens — an analytical estimate, not an experimental measurement. Fig. 6 plots this analytical rate against the context lengths used by each configuration; it describes the 3B-class model in Table I, not the Llama-3.2-1B-Instruct model used in the completed run of Section VII.

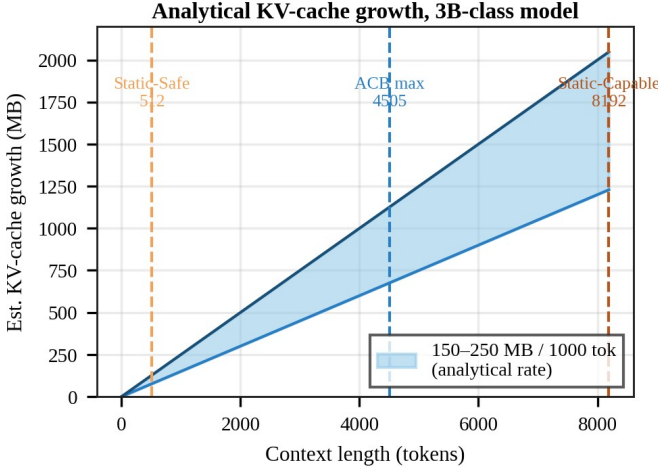


Fig. 6. Analytical KV-cache growth for the 3B-class model (Section V-B) at the stated rate of 150–250 MB per 1000 context tokens. This is a design-time estimate, not a measurement from the completed run. Dashed lines mark the context values used by each configuration in Table II.

C. Controlled Experimental Protocol

The final controlled run used Llama-3.2-1B-Instruct and a fixed 20-query subset of the planned workload, with query identifiers 0–19, drawn from short_factual, multi_turn, and long_context categories. The same sequence was executed by Static-Safe, Static-Capable, and ACB, all with a 128-token generation cap. This cap was a documented protocol deviation from the originally planned 2048-token cap, made because Static-Capable was prohibitively slow during smoke testing at the larger value; applying the same reduced cap to all three configurations preserved the matched workload. Similarly, the original protocol specified 200 mixed queries, but the completed run used the documented 20-query subset for the same reason. Server readiness was checked via a health endpoint before every query, and memory telemetry was collected at approximately 250-ms intervals. The final controlled dataset contains 60 observations — 20 per configuration. Table II lists the three configurations.

TABLE II. EXPERIMENTAL CONFIGURATIONS

Configuration	Quant.	Context	Gen. cap	Threads
Static-Safe	Q4_K_M	512	128	4
Static-Capable	Q8_0	8192	128	4
ACB	Q4_K_M	Dynamic	Dynamic	7

Fig. 7 visualizes the context and thread settings of Table II directly, alongside the context range ACB actually selected (Table V).

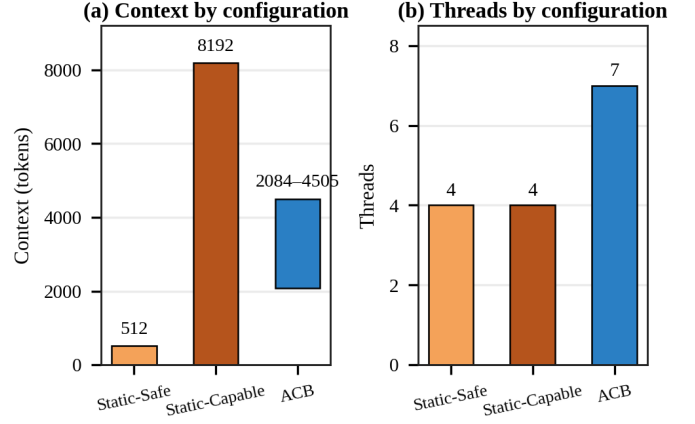


Fig. 7. Configuration comparison. (a) Context length: fixed for Static-Safe and Static-Capable, and the observed min–max range ACB selected across the 20 queries (Table V). (b) CPU thread count per configuration (Table II).

VI. EXPERIMENTAL INSTRUMENTATION AND DATA VALIDATION

A recurring methodological concern in this paper is separating measured observations from analytical or design-time quantities. Rather than repeat that caveat after every number, I state it once here: unless explicitly marked as an analytical estimate (Section V-B) or a derived/summary statistic, every figure in Sections VI–VII is a directly recorded harness observation, auditable from the raw per-query CSV.

A. Measurement Scope

The completed experiment distinguishes three classes of information: (i) runtime observations recorded by the inference harness — generated-token count, end-to-end latency, recorded tokens-per-second, memory state, memory-pressure state, thread count, context limit, HTTP status, and validity status; (ii) the controller's selected configuration for each ACB request; and (iii) the analytical memory model of Section V, which is retained as a design model and never substituted for runtime telemetry.

The controlled run used the same workload ordering for each configuration. Query identifiers 0–19 were executed sequentially by Static-Safe, Static-Capable, and ACB; only the configuration changed between experimental blocks. Server readiness was checked before each request so that no request was issued to a server that had not reported the expected healthy state. Fig. 8 summarizes this harness pipeline.

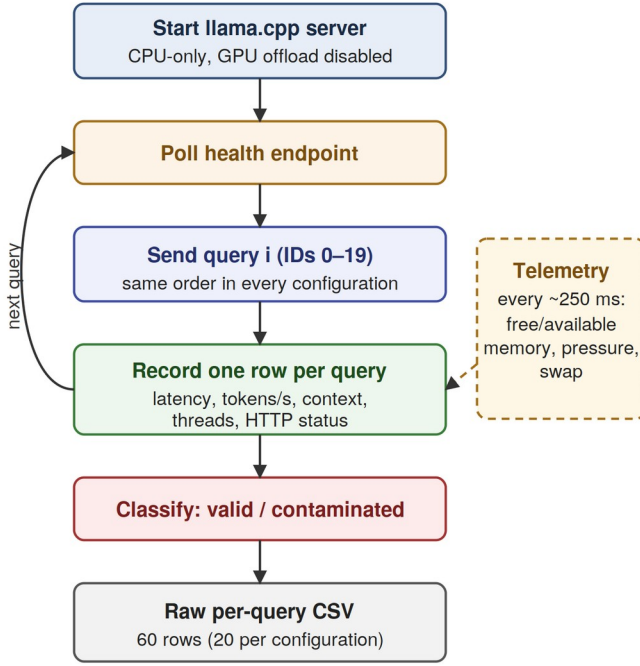


Fig. 8. Evaluation harness. Readiness is polled before every query, telemetry is sampled in the background, and each request produces one classified row in the raw CSV.

B. Telemetry and Harness Fields

For each request, the harness recorded the configuration name, model, quantization, query type and identifier, context limit, generated tokens, latency, recorded throughput, free and available memory, memory-pressure state, swap usage, thread count, KV policy, truncation state, process-failure state, HTTP status, error code, and data-status classification — sufficient to audit any individual observation without reconstructing it from summary statistics. The reported swap value is a point-in-time reading, not a cumulative count of storage reads or writes, and recorded free memory is not treated as an OOM indicator by itself; execution status and memory-pressure state are considered together.

C. Readiness and Contamination Control

An earlier V1 harness generated 13 contaminated observations because requests were issued before model readiness; those rows are excluded from the completed dataset. The V2 harness introduced explicit readiness polling and dispatched measured requests only after the server returned the expected healthy response. The completed controlled run contains no rows classified as contaminated. A final audit found exactly 20 rows per configuration (60 total), identical query identifiers and ordering across configurations, and no rows with process failure or truncation. One recorded throughput value differs slightly from a value recomputed from the CSV’s rounded latency field, because the stored latency is displayed at lower precision than the internal timing value; this does not affect the validity classification of that observation.

D. Interpretation of Extreme Observations

The 4057.91-s Static-Capable observation is retained because the request completed successfully and was not classified as a startup failure, HTTP failure, truncation, process termination, or OOM event. It occurred on run ID 21, the first Static-Capable query, using Q8_0 quantization and an 8192-token context limit; the raw telemetry also recorded substantial swap usage. These facts establish the observed execution conditions, but the present single-trial harness does not isolate the individual contributions of CPU scheduling, storage traffic, memory reclamation, or other OS mechanisms — nor does it rule out incidental confounds such as thermal throttling or background-process contention, which I did not independently instrument in this run. The same non-removal principle applies to unusually fast observations: no outlier was manually removed from the final dataset, since doing so after observing the outcome would bias the sample.

E. Controlled-Run Configuration

Table III records the exact configuration used for the completed workload.

TABLE III. COMPLETED CONTROLLED-RUN CONFIGURATION

Parameter	Recorded configuration
Hardware	Apple M2 MacBook Pro, 8 GB unified memory
Inference	llama.cpp b3799, CPU-only
GPU offload	Disabled with --n-gpu-layers 0
Model	Llama-3.2-1B-Instruct
Workload	Query IDs 0–19
Queries / configuration	20
Generation cap	128 tokens
Static-Safe	Q4_K_M, context 512, 4 threads
Static-Capable	Q8_0, context 8192, 4 threads
ACB	Q4_K_M, dynamic context, 7 threads
Readiness	HTTP health check before each query
Telemetry	psutil + macOS memory pressure

VII. EXPERIMENTAL RESULTS

This section reports observations from the completed controlled run only; analytical estimates and smoke-test measurements are excluded.

A. Dataset Validity

The final dataset contained exactly 20 observations per configuration across the same 20 query identifiers, in the same sequence. All 60 rows were classified valid, with process_failure=False and truncated=False for every row (Table IV).

TABLE IV. CONTROLLED-RUN VALIDITY

Configuration	Queries	Valid	Failed	Trunc.	OOM
Static-Safe	20	20	0	0	0
Static-Capable	20	20	0	0	0
ACB	20	20	0	0	0

B. Observed Latency and Throughput

Among individual recorded observations, Static-Safe included a 402.42-s latency observation, Static-Capable included a 4057.91-s latency observation, and ACB's longest recorded latency was 53.14 s. These are individual recorded query observations, not model-derived estimates. The same records show individual throughput observations spanning 0.32–67.93 tokens/s for Static-Safe, 0.03–45.75 tokens/s for Static-Capable, and 0.16–56.48 tokens/s for ACB — reported only as the observed endpoints present in the raw dataset, with no additional values inferred. The 4057.91-s Static-Capable observation occurred on run ID 21, the first Static-Capable query, and completed as a valid, non-truncated request. Fig. 9 plots the longest observed latency and the observed throughput range for each configuration; panel (a) shows one maximum per configuration from a single trial, not a mean or median.

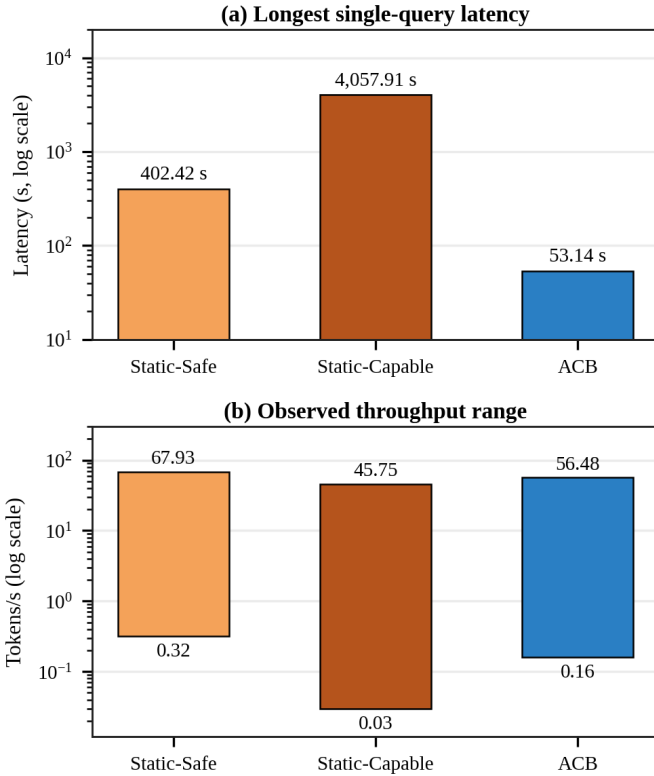


Fig. 9. Recorded extremes per configuration (20 queries each, single trial): (a) longest observed single-query latency; (b) observed range of per-query throughput, showing only the recorded minimum and maximum.

C. Memory Telemetry

The raw dataset recorded point-in-time free memory, available memory, swap usage, and memory-pressure state for each query. Mean values are intentionally not reproduced here,

as they are derived statistics rather than direct measurements. Zero abnormal pressure flags were recorded at query completion for all three configurations, and no process termination or OOM event occurred. The Static-Capable 4057.91-s observation coincided with the high-memory-demand Q8_0 configuration and severe swap use; this establishes an association between the high-memory configuration and severe latency in this run, without establishing a full causal decomposition.

D. ACB Controller Adaptation

Table V summarizes controller state across all 20 ACB queries: short-factual queries received context values between 2084 and 2772 tokens, multi-turn queries received 4173 tokens, and long-context queries received 4505 tokens. All 20 ACB observations used seven CPU threads — the workload demonstrates context adaptation but does not exercise thread-count adaptation. Fig. 10 shows the context selected for each of the 20 queries against the two static settings.

TABLE V. OBSERVED ACB CONTEXT SELECTION

Query type	Observed context	Threads
short_factual	2084–2772	7
multi_turn	4173	7
long_context	4505	7

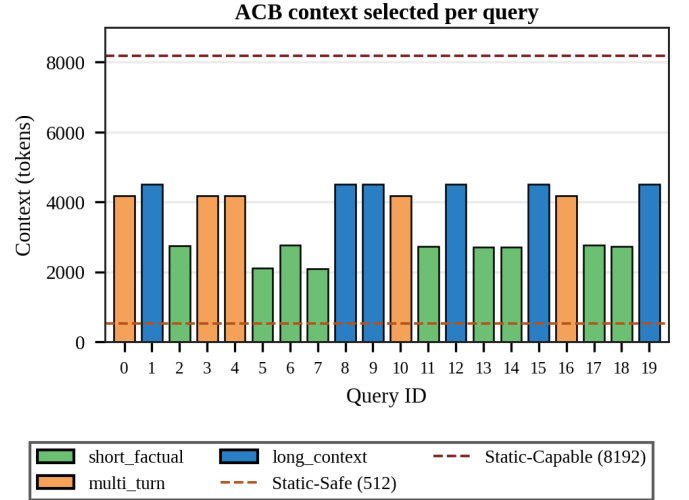


Fig. 10. Context budget selected by ACB for each query (from Table VI), colored by query type; dashed lines mark the fixed contexts of the two static configurations.

E. Measured Reliability

No query in the controlled dataset was marked truncated. All 60 requests completed with HTTP status 200 and valid-data classification, and no process termination or OOM event was recorded. These observations establish execution reliability for this workload and environment only; they do not establish reliability under other models, devices, background loads, or longer generations.

VIII. DISCUSSION

The completed run provides an empirical test of the ACB control layer under a constrained CPU-only environment. The clearest observation is that the controller selected different context budgets for different query classes while all 20 ACB requests completed without recorded termination or truncation. ACB's longest individual latency was 53.14 s in this run, versus 402.42 s and 4057.91 s for Static-Safe and Static-Capable, respectively — supporting the narrower claim that the tested ACB configuration avoided the extreme latency observations encountered by both static configurations on this workload.

Two aspects of this comparison deserve explicit caution. First, Static-Capable's fixed context (8192 tokens) is substantially larger than any context ACB selected (2084–4505 tokens); part of ACB's latency advantage may therefore reflect that ACB never chose the most extreme setting available, rather than a general adaptive-control benefit. A static baseline fixed near ACB's typical operating context (~4000 tokens) would isolate these two effects, and I specify such a baseline in Appendix B. Second, thread count was constant (seven) across all ACB observations in this run, so the experiment does not exercise or validate thread-count adaptation — one of the five knobs ACB is designed to control.

The throughput observations do not support a claim that ACB universally maximizes token throughput: Static-Safe contains the highest individual throughput observation (67.93 tokens/s), against ACB's highest individual observation (56.48 tokens/s). The contribution demonstrated here is better characterized as adaptive resource allocation and bounded latency behavior than as maximum raw decoding speed.

The Static-Capable 4057.91-s observation is especially relevant to memory-constrained deployment. It completed successfully and should not be treated as an OOM failure, but the raw telemetry records substantial swap usage during the request. The data establish an association between the high-memory configuration and severe latency in this run; further instrumentation — e.g., independently tracking thermal state and background-process activity — would be required to attribute the delay fully to a specific OS mechanism, and I did not rule out such confounds in the present single-trial protocol.

Finally, although the ACB architecture exposes five resource knobs, the completed workload did not cause all five to vary: threads were fixed at seven, and the empirical run used one model and one quantization level for ACB. The experiment therefore demonstrates the controller's context-budget adaptation but does not independently validate every proposed control dimension.

IX. LIMITATIONS

This study has several limitations that bound how far its results can be read. The controlled workload contained 20 queries per configuration rather than the 200 I originally planned, and I reduced the generation cap to 128 tokens for every configuration because Static-Capable was prohibitively slow at longer lengths. Each configuration was run in a single trial, so I could

not measure between-run variance, and every latency figure reported here comes from one execution. All measurements come from one Apple M2 system with 8 GB of unified memory running a single model, Llama-3.2-1B-Instruct, and the whole study was carried out by one author on that one machine.

Several parts of the design are also untested. I did not include a synthetic background-load experiment, even though competing memory demand is exactly the situation ACB is built for. The experiment does not isolate the causal contribution of each control variable, and thread count was never exercised because ACB used seven threads on every query. The complexity estimator $s(q)$ was not validated against ground-truth query difficulty or measured memory demand, and the swap telemetry is a point-in-time reading rather than a cumulative measure of storage traffic. Appendix B proposes the experiments needed to close these gaps.

X. CONCLUSION

In this paper I introduced Adaptive Cognitive Budgeting, a control layer that combines query-complexity estimation and host-resource telemetry to allocate inference resources for GPU-free LLM execution on memory-constrained personal computers. The completed controlled experiment evaluated 20 matched queries for each of Static-Safe, Static-Capable, and ACB on an Apple M2 system with 8 GB unified memory using CPU-only Llama-3.2-1B-Instruct inference.

All 60 controlled requests completed without recorded process termination, truncation, or OOM events. The ACB controller selected context budgets between 2084 and 2772 tokens for short-factual queries, 4173 tokens for multi-turn queries, and 4505 tokens for long-context queries. The longest ACB latency observation was 53.14 s, compared with individual observations of 402.42 s for Static-Safe and 4057.91 s for Static-Capable. These observations demonstrate the behavior of the implemented controller on the evaluated machine and workload; they do not establish universal superiority or generalization to other devices, models, or workloads.

Appendix B specifies the repeated-trial, additional-baseline, background-load, and estimator-ablation protocol required to test the repeatability and generality of the observed behavior before extending these claims further.

APPENDIX A. CONTROLLER TRACE AND DATA AUDIT

Table VI reports the controller state extracted from the 20 ACB observations — recorded query type, selected context, and selected thread count — so the adaptive behavior is auditable at the query level rather than only through aggregate summaries. The trace shows context selection was not fixed: 4173 tokens for every multi-turn query, 4505 tokens for every long-context query, and several distinct smaller values for short-factual queries. Thread count remained seven for every recorded ACB request. Table VII summarizes the checks applied to the raw CSV.

TABLE VI. PER-QUERY ACB CONTROLLER STATE (ALL 20 QUERIES USED 7 THREADS)

ID	Type	Ctx	ID	Type	Ctx
0	multi_turn	4173	10	multi_turn	4173
1	long_context	4505	11	short_factual	2736
2	short_factual	2748	12	long_context	4505
3	multi_turn	4173	13	short_factual	2699
4	multi_turn	4173	14	short_factual	2699
5	short_factual	2109	15	long_context	4505
6	short_factual	2760	16	multi_turn	4173
7	short_factual	2084	17	short_factual	2772
8	long_context	4505	18	short_factual	2736
9	long_context	4505	19	long_context	4505

TABLE VII. RAW DATASET AUDIT

Audit item	Verified result
Total controlled rows	60
Static-Safe / Static-Capable / ACB rows	20 / 20 / 20
Query IDs per configuration	0–19
Query ordering	Identical across configurations
Valid observations	60
Contaminated observations	0
Process failures	0
Truncated outputs	0
HTTP status	200 for all controlled requests
OOM events	0 recorded
Abnormal pressure flags	0 at query completion

The empirical claim in this paper is deliberately bounded: one Apple M2 system, 8 GB unified memory, one model, CPU-only llama.cpp inference, one 20-query workload subset, a 128-token generation cap, and one trial per configuration. The candidate-model weight sizes and KV-cache growth expressions (Section V-B) remain analytical design-time quantities and are kept separate from the measured Llama-3.2-1B-Instruct observations reported in Sections VI–VII throughout the paper, so that feasibility estimates are never mistaken for testbed measurements. The raw per-query CSV, controller trace, harness schema, and workload identifiers described here are sufficient to reconstruct the completed experiment and should be retained together with this paper for independent audit.

APPENDIX B. EXTENDED EVALUATION PROTOCOL (PROPOSED)

The completed experiment in Sections V–VII establishes a single audited trial per configuration. The following protocol extensions are proposed to test the repeatability and generality of the observed behavior; none of the results below have yet been collected, and none of the claims in Sections VII–VIII

depend on them. Fig. 11 shows how each proposed experiment extends the completed run.

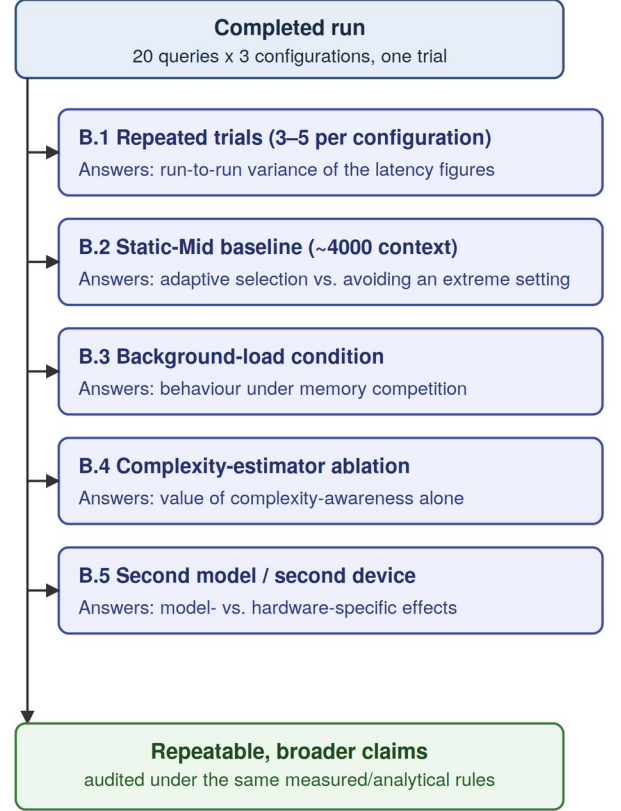


Fig. 11. Proposed extended evaluation (Appendix B). Each experiment answers one open question left by the single-trial run.

B.1. Repeated Trials

Repeat the full 20-query workload 3–5 times per configuration on the same hardware, holding model, workload, and cap fixed. This converts the single-point latency and throughput figures in Tables IV–V into medians and ranges, and directly addresses the largest generalization gap in the completed run: every headline latency number in Sections VII–VIII currently rests on $n = 1$.

B.2. Static-Mid Baseline

Add a fourth configuration, Static-Mid (e.g., Q4_K_M or Q8_0 at a fixed context near ACB's typical operating range, ≈ 4000 tokens, four threads). Comparing ACB against Static-Mid, in addition to Static-Safe and Static-Capable, isolates the benefit of adaptive per-query selection from the benefit of simply avoiding the most extreme fixed configuration (Static-Capable's 8192-token context).

B.3. Background-Load Condition

Introduce a synthetic memory-consuming background process (e.g., a controlled allocator holding a fixed fraction of system memory) running concurrently with each configuration's workload. This targets the exact deployment scenario ACB is designed for — competition for shared memory with other applications — which the completed run does not exercise.

B.4. Complexity-Estimator Ablation

Run ACB with (a) the real complexity estimator $s(q)$ and (b) a fixed or randomized substitute score, holding the telemetry-driven memory cap constant. Comparing these two conditions isolates how much of ACB's observed benefit comes from complexity-awareness specifically, versus from memory-awareness alone — the estimator is currently the least independently evaluated component of the architecture (Section IV-A).

B.5. Cross-Model / Cross-Device Generalization

Repeat the protocol with a second model (e.g., Qwen2.5-3B-Instruct) on the same machine, and with the original Llama-3.2-1B-Instruct model on a second 8 GB-class machine, to separate model-specific from hardware-specific contributions to the observed latency and context-selection behavior.

B.6. Reporting Standard for Extended Results

Any results collected under B.1–B.5 should follow the same measured/analytical separation, contamination control, and non-removal-of-outliers principles used in Sections VI and Appendix A, so that extended results remain directly comparable to — and auditable against — the completed run reported in this paper.

REFERENCES.

- [1] G. Gerganov et al., “llama.cpp: LLM inference in C/C++,” GitHub repository, 2023–2026. Available: <https://github.com/ggml-org/llama.cpp>
- [2] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” arXiv:2210.17323, 2022.
- [3] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, “AWQ: Activation-aware weight quantization for LLM compression and acceleration,” arXiv:2306.00978, 2023.
- [4] J. Lei and S. Ilager, “ARKV: Adaptive and resource-efficient KV cache management under limited memory budget for long-context inference in LLMs,” arXiv:2603.08727, 2026.
- [5] “Parallel CPU-GPU execution for LLM inference on constrained GPUs (APEX),” arXiv:2506.03296, 2025.
- [6] Y. Song, Z. Mi, H. Xie, and H. Chen, “PowerInfer: Fast large language model serving with a consumer-grade GPU,” arXiv:2312.12456, 2023.
- [7] D. Saikumar and B. Varghese, “NeuroFlux: Memory-efficient CNN training using adaptive local learning,” arXiv:2402.14139, 2024.
- [8] S. Huang, H. Wang, W. Zhong, Z. Su, J. Feng, B. Cao, and Y. R. Fung, “AdaCtrl: Towards adaptive and controllable reasoning via difficulty-aware budgeting,” arXiv:2505.18822, 2025.
- [9] Z. Li, Q. Dong, J. Ma, D. Zhang, K. Jia, and Z. Sui, “SelfBudgeter: Adaptive token allocation for efficient LLM reasoning,” arXiv:2505.11274, 2025.
- [10] “Thinking economically: A hierarchical framework for adaptive-complexity reasoning in large language models,” arXiv:2606.01168, 2026.
- [11] “Cognitive load-aware inference: A neuro-symbolic framework for optimizing the token economy of large language models,” arXiv:2507.00653, 2025.
- [12] “Learning how hard to think: Input-adaptive allocation of LM computation,” arXiv:2410.04707, 2024.
- [13] “Cognitive edge computing: A comprehensive survey on optimizing large models and AI agents for pervasive deployment,” arXiv:2501.03265, 2025.
- [14] A. Toktassyn and J.-G. Park, “Efficient PEFT methods with adaptive checkpointing for vision models and VLMs on resource constrained consumer-GPUs,” arXiv:2607.02158, 2026.